Transferência de arquivos P2P

Sistemas Distribuídos - TP2 | CEFET-MG | 2026/2
Professora: Michelle Hanne.
Integrantes: Augusto Coelho Fagundes e ADJAILSON FREIRE DE SÁ FILHO.
30/10/2026.

Código-fonte: https://github.com/Augusto42/sd-tp2-p2p-2026

1. Arquitetura e decisões

Cada peer é um processo Python com um servidor TCP e tarefas clientes no mesmo event loop asyncio. Ele atende solicitações enquanto baixa de seus vizinhos. A biblioteca padrão reduz dependências e permite concentrar o trabalho em protocolo, concorrência e integridade.

Os vizinhos são estáticos, configurados por IP e porta. Não foi usado tracker, pois é opcional. A matriz usa malha completa: com quatro peers, cada um configura os outros três como vizinhos. Cada leecher estabelece contato com todos, mesmo se um download curto concluir antes da conexão com um vizinho que iniciou depois.

O arquivo é dividido logicamente em blocos de 1.024 ou 4.096 bytes. O metadado JSON informa versão, tamanho, tamanho de bloco, nome informativo, hashes por bloco e hash completo. Todos recebem esse metadado previamente; ele é considerado confiável.

Os blocos ficam em um arquivo parcial em disco, gravados por offset, evitando carregar todo o arquivo em memória. O conjunto have registra blocos verificados; busy reserva pedidos em voo. Prioridades circulares de índices, baseadas na porta local, favorecem blocos diferentes entre leechers e seu compartilhamento.

Após cada bloco passar na checagem de tamanho e SHA-256, ele fica disponível para upload. Ao concluir, verifica-se o SHA-256 completo e renomeia-se o parcial para arquivo_final.bin. O peer continua servindo; executar.py encerra automaticamente todos os processos quando os downloads ativos terminam, inclusive na demonstração em um único terminal.

2. Protocolo e implementação

TCP fornece um fluxo de bytes, não mensagens. Por isso cada frame usa prefixo de comprimento de 4 bytes big-endian e corpo de até 8 MiB. readexactly trata frames divididos em múltiplas leituras. Conexões persistentes evitam reconectar a cada bloco.

Mensagem	Conteúdo / função
have (JSON)	Pedir lista dos índices disponíveis
get (JSON)	Pedir índice de bloco e identificar arquivo pelo hash
J + JSON	Responder inventário de blocos
B + bytes	Entregar bloco binário sem base64
E + texto	Indicar arquivo incorreto ou bloco indisponível

Um pedido fica em voo por conexão, dispensando identificador de correlação. Cada vizinho possui sua própria tarefa de download. O servidor mantém uma tarefa por conexão recebida. Gravação/atualização do conjunto e leitura do bloco não contêm await; assim, outras tarefas não alteram o ponteiro compartilhado entre seek e read/write.

Conexão tem timeout de 2 s; resposta, 5 s; servidor ocioso, 15 s. Falhas liberam reservas e tentam novamente após 100 ms; falta de blocos novos gera nova consulta após 20 ms. Estes valores privilegiam uma demonstração em rede local. Não há limite global de espera.

Hashes por bloco rejeitam dados corrompidos antes de publicá-los. O hash final confirma remontagem. O nome do metadado é informativo: a saída usa nome local fixo, evitando usar dados remotos como caminho de escrita.

3. Procedimento experimental

Executaram-se 24 configurações, com cinco repetições cada (120 transferências): 2/4 peers x 1.024/4.096 bytes x 10/20 KiB, 1/5/10/20 MiB. KB e MB foram interpretados como unidades binárias. Todos usam processos independentes, malha completa e TCP loopback.

A bateria é automática: **python3 executar.py --testes --repeticoes 5 --seed 2026**. Geração pseudoaleatória reproduz os mesmos bytes por tamanho; a ordem dos casos é embaralhada por repetição com semente registrada. Cada caso verifica tamanho, SHA-256, índices, fontes e conexão a todos os vizinhos. Logs e metadados são preservados; binários temporários são descartados.

Média aritmética e desvio-padrão amostral (denominador n - 1) são calculados dos cinco tempos. O intervalo medido vai da inicialização até arquivos completos e contato com todos os vizinhos, incluindo logs. Portas, caches e escalonamento variam; os números não representam capacidade de enlace nem desempenho em LAN/WAN.

4. Resultados da matriz completa

Arquivo	2 peers / 1 KiB	2 peers / 4 KiB	4 peers / 1 KiB	4 peers / 4 KiB
10 KiB	0.446 ± 0.035	0.399 ± 0.066	0.663 ± 0.072	0.691 ± 0.070
20 KiB	0.393 ± 0.050	0.417 ± 0.066	0.687 ± 0.091	0.664 ± 0.065
1 MiB	0.560 ± 0.053	0.507 ± 0.101	0.835 ± 0.101	0.685 ± 0.117
5 MiB	1.276 ± 0.175	0.722 ± 0.077	2.487 ± 0.572	1.082 ± 0.194
10 MiB	1.964 ± 0.135	0.813 ± 0.038	4.678 ± 1.094	1.603 ± 0.274
20 MiB	4.050 ± 0.794	1.343 ± 0.193	9.618 ± 1.681	2.696 ± 0.577

Tempo em segundos: média ± desvio-padrão amostral; n = 5 em cada célula. Cada configuração representa cinco execuções completas, com os mesmos critérios de conclusão.

Integridade: 120/120 execuções aprovadas. Todos os arquivos finais tiveram tamanho e SHA-256 iguais ao original. Os logs confirmam que cada leecher se conectou a todos os vizinhos configurados. Cada bloco foi recebido uma única vez e sua fonte pertence à lista de vizinhos.

Os valores acima são medições da malha completa. Quatro peers entregam três cópias em paralelo, com possibilidade de redistribuição entre leechers. Os tempos incluem inicialização, cópia da semente, espera pelo contato com todos os vizinhos, polling e logs; não medem apenas transmissão de bytes.

Dados individuais em resultados/medicoes.json; estatísticas em resultados/estatisticas.csv e .json; ambiente e sementes em resultados/ambiente.json. Cada linha aponta para logs e manifesto compartilhado, permitindo conferir os cinco valores e recalcular média/desvio. Nenhum tempo de cenário com atraso artificial integra a tabela acima.

Os cenários adicionais comprovam redistribuição antes da conclusão, início tardio da semente, múltiplas fontes e recuperação após falha. A demonstração automática é **python3 executar.py**: cria a rede, provoca falhas, verifica integridade e encerra todos os processos, sem abrir terminais adicionais.

5. Falhas e recuperação

Queda temporária: única semente interrompida por SIGKILL durante downloads incompletos. Leechers preservaram blocos e reconectaram após seu retorno: 36 tentativas e 2.671 s entre queda e conclusão. **Queda permanente controlada:** duas sementes iniciais; uma foi morta sem retorno, e os três sobreviventes concluíram em 1.735 s após a queda. Tamanho e SHA-256 corretos em ambos.

Peer aleatório offline: cinco ensaios, quatro peers em malha e duas sementes iniciais (A/B). O ID foi sorteado entre os quatro peers com sementes 0 a 4. A falha foi aplicada após observar envio real pelo peer escolhido e downloads sobreviventes incompletos. Novos blocos vieram de outras fontes, sem reiniciar o peer morto.

Sorteio	Peer morto	Blocos novos	Fontes após queda	Tempo (s)	SHA-256
0	D	402	C: A/B	0.830	OK
1	B	975	C: A/D; D: A/C	1.749	OK
2	A	911	C: B/D; D: C/B	1.394	OK
3	B	968	C: D/A; D: A/C	1.654	OK
4	B	926	C: A/D; D: A/C	1.141	OK

Cada linha é um ensaio de falha (n = 1), não uma média de desempenho. Blocos novos soma os recebidos pelos leechers sobreviventes após a queda; fontes excluem o peer morto. Tempo vai do SIGKILL à conclusão dos ativos. Arquivo: 524.325 bytes, blocos de 1 KiB; atrasos artificiais tornam a falha observável. Evidências em resultados/queda_aleatoria_N/controle.json e peer_N.jsonl.

Outras verificações: 83 envios por leechers antes de completar no cenário parcial; semente tardia e malha aprovadas. **11/11 unitários**: remontagem fora de ordem, último bloco, corrupção, tamanho, arquivo vazio, metadados, hashes, frames fragmentados/truncados/excessivos e conexão após conclusão curta.

6. Conclusão e limites

130 integrações e 11 unitários aprovados. A malha permite fontes alternativas e o executor automatiza início, falha, verificação e encerramento. Sem outra cópia de um bloco perdido, é necessário aguardar sua fonte. Reiniciar o próprio leecher ainda descarta seu parcial. Os testes mantêm os leechers sobreviventes ativos; não demonstram retomada persistente nem tolerância a perda simultânea de todas as cópias.

Referências: enunciado SD_TP2_2_2026.pdf, pp. 1-2; código e evidências no repositório indicado. Ensaios locais com metadado confiável; não há autenticação, descoberta automática ou medição em rede remota.